

GRAPH SEARCH ALGORITHM VISUALIZER (GSAV)

BY – Atharva Kulkarni

Rgno.:25BAI10584

B.tech CSE AIML

to – Garima Jain

Course – Introduction to AI / ML

Date – 31th March 2026

Introduction

The project is an interactive visualization tool for understanding graph search algorithms, developed in Python using Jupyter Notebook. It allows users to select a start node, goal node, and algorithm, and then visually observe how the algorithm explores the graph step-by-step.

The system supports multiple algorithms and provides both graphical output and traversal logs, helping users understand the internal working of each algorithm clearly.

Problem Statement

Students often find it difficult to understand how search algorithms work internally, especially concepts like node expansion, frontier management, and path selection.

Traditional learning methods lack interactivity and visualization.

There is a need for a system that visually demonstrates these algorithms in an intuitive and interactive way.

Functional Requirements

1. Select start and goal nodes
2. Choose algorithm (BFS, DFS, UCS, A*)
3. Visualize graph traversal step-by-step
4. Highlight visited, frontier, and path nodes
5. Display final path and cost
6. Show traversal logs for each step
7. Support weighted graphs
8. Provide heuristic-based search (A*)

Non-functional Requirements

Requirement | Description

Usability | Interactive and easy-to-use UI

Reliability | Accurate algorithm execution

Performance | Fast rendering and updates

Scalability | Can be extended to larger graphs

Portability | Runs on Jupyter Notebook / Python environments

System Architecture

The system follows a modular architecture suitable for an interactive notebook-based application:

- Presentation Layer (UI/UX):

Implemented using ipywidgets and matplotlib for interactive controls and visualization.

- Application Logic Layer:

Implements search algorithms such as BFS, DFS, UCS, and A*. Handles traversal logic and step tracking.

- Data Layer:

Graph data is stored in adjacency list format with predefined nodes, edges, and weights.

DESIGN DIAGRAMS

Use Case Diagram

Overview: This diagram represents the user interaction with the system.

Actors: User

Use Cases: Select Start Node, Select Goal Node, Choose Algorithm, Run Visualization, View Graph, Analyze Logs

Workflow Diagram

Process Flow:

Start Application → Select Nodes & Algorithm → Click Run
→ Execute Algorithm → Generate Steps → Display
Visualization → Show Traversal Log → End

Class Diagram

Main Components:

Graph Module

Responsibilities: Store nodes, edges, and weights

Algorithm Module

Functions: bfs(), dfs(), ucs(), astar()

Visualization Module

Responsibilities: Draw graph, highlight nodes, update steps

UI Module

Responsibilities: Handle user input, trigger execution, display output

Design Decisions & Rationale

Design Choice | Reason

NetworkX | Efficient graph handling and visualization

Matplotlib | Clear and customizable graph rendering

ipywidgets | Interactive UI in Jupyter Notebook

Euclidean Heuristic | Simple and effective for A*

Step Tracking | Enables detailed visualization

Implementation Details

The project consists of the following components:

- Graph Construction – Builds adjacency list from edges
- Search Algorithms – BFS, DFS, UCS, A* implemented

with step tracking

- Visualization Engine – Displays graph states using colors
- UI System – Allows user interaction via widgets

The application dynamically updates the graph and logs based on user input.

Testing Approach

Test Type | Performed On

Unit Test | Individual algorithms (BFS, DFS, UCS, A*)

UI Test | Widget interactions and button actions

Visualization Test | Graph rendering and color updates

Logic Test | Correctness of path and cost

Error Handling | Invalid input cases (same start/goal)

Challenges Faced

- Managing different data structures for each algorithm
- Keeping visualization clear without clutter
- Synchronizing UI updates with algorithm execution

- Designing an effective heuristic for A*
- Handling step tracking uniformly across algorithms

Learnings & Key Takeaways

- Strong understanding of search algorithms
- Implementation of priority queues and heuristics
- Graph visualization using Python libraries
- Building interactive UI in Jupyter
- Debugging complex algorithm flows

Future Enhancements

- Add more algorithms (Greedy, Bidirectional Search)
- Allow custom graph input
- Add animation instead of static steps
- Convert into web application (Streamlit/Flask)
- Add performance comparison metrics

References

- Python Official Documentation
- NetworkX Documentation
- Matplotlib Documentation
- AI Algorithm Tutorials (GeeksForGeeks, etc.)